

# Aws Deployment Guide

### Table of Contents

- AWS Deployment Guide
  - Services
  - Network
  - Security Groups
  - Environment Variables
  - Deployment Steps
  - HTTPS Recommendation
  - Google OAuth Origins
  - React Frontend Build on EC2
  - Cache Management Deployment Checks
  - Python Analytics API EC2

# AWS Deployment Guide

Do not use Render or Netlify. The target deployment uses EC2 plus Amazon RDS PostgreSQL.

### Services

| Service                         | Port   | Responsibility                                                                      |
|---------------------------------|--------|-------------------------------------------------------------------------------------|
| Frontend EC2 + Nginx            | 80/443 | Serve landing page and private web app static files.                                |
| Node Application API<br>EC2/ASG | 3000   | Auth/session, scoped academic operations, leads, finance, rules, reports and audit. |
| Python Analytics API<br>EC2     | 8000   | Attendance risk, scholarship readiness, branch performance and workload analytics.  |
| Amazon RDS<br>PostgreSQL        | 5432   | Normalized relational database.                                                     |

The current executable is one Express application. Deploy it as one application target (with multiple identical instances behind an ALB for availability), not as pretend services that each still expose every route. A future physical split requires separate entry points, least-privilege database identities and independent health/deployment policies first.

## Network

- VPC with public subnets for ALB/Nginx.
- Private app subnets for API EC2 instances.
- Private database subnets for RDS.
- Optional Application Load Balancer terminates HTTPS with ACM certificate.

## Security Groups

| Group         | Inbound                                                                        |
|---------------|--------------------------------------------------------------------------------|
| ALB           | <code>443</code> from internet, optional <code>80</code> redirect to HTTPS.    |
| Frontend EC2  | <code>80/443</code> only from ALB.                                             |
| Node API EC2  | <code>3000</code> only from ALB or frontend security group.                    |
| Analytics EC2 | <code>8000</code> only from ALB or frontend security group.                    |
| RDS           | <code>5432</code> only from Node API and Python Analytics API security groups. |

## Environment Variables

- `NODE_ENV=production`
- `DB_DRIVER=prisma`
- `DATABASE_URL=postgres://...`
- `SESSION_SECRET`
- `SESSION_TTL_MINUTES`
- `JWT_ISSUER=american-latin-class-auth`
- `JWT_AUDIENCE=american-latin-class-services`
- `GOOGLE_CLIENT_ID`
- `ALLOW MOCK GOOGLE TOKENS=false`
- `POSTMAN_LOGIN_ENABLED=false` (startup rejects this flag outside automated tests).
- `EXPOSE_SESSION_TOKEN=false` (startup rejects response-token exposure in staging/production).
- `SEED_*_PASSWORD` only when running the temporary role-test seed; rotate or remove the seeded credentials before a real production handoff.

- `CORS_ORIGINS=https://app.americanlatinclass.edu`
- `TRUST_PROXY=true` only behind the controlled ALB/Nginx proxy.
- `COOKIE_SECURE=true`
- `FORCE_HTTPS=true`
- `AUTH_RATE_LIMIT_MAX=20`
- `JSON_BODY_LIMIT_BYTES=102400`
- `AWS_REGION`

Python Analytics API variables:

- `DATABASE_URL=postgres://...`
- `SESSION_SECRET` with the same value used by the Node Application API.
- `ANALYTICS_AUTH_REQUIRED=true`
- `ANALYTICS_CORS_ORIGINS=https://academy.example.invalid` (replace through deployment secrets)
- `ANALYTICS_SERVICE_NAME=American Latin Class Analytics API`

Store secrets in AWS Systems Manager Parameter Store or Secrets Manager and grant the instance/task role only the exact parameters it needs.

Production/staging processes fail fast if `SESSION_SECRET` is missing/default/short, `DATABASE_URL` is missing, mock Google tokens are enabled, token exposure is enabled or an unsafe authentication fallback is configured.

Branch directors must be assigned to one or more branches through `/api/v1/users/{id}/branch-access`; otherwise their role authenticates successfully but has no branch-scoped data visibility.

## Deployment Steps

1. Create private RDS PostgreSQL with encryption, automated backups/PITR, deletion protection and a security group reachable only from application roles.
2. Build in CI or a controlled build host with `npm ci`, `npm test`, `npm run db:generate` and `npm run frontend:build`.
3. Snapshot the target database and run `npm run db:migrate:status`, then `npm run db:migrate`. The runner applies ordered migrations `001` through `012` using a lock, checksums and transactions; never invoke a schema push.
4. For a new database, run the one-time `npm run db:bootstrap-admin`, log in, rotate its temporary password and immediately remove `BOOTSTRAP_ADMIN_*` from the environment. Create real branch records through the controlled application workflow. Do not run demo/role seeds in production.
5. Run `npm run db:smoke:prisma` and `npm run db:smoke:operational` against the migrated target before accepting traffic.
6. Deploy source plus the already-built `dist/frontend`. If build and runtime share one host, run `npm ci`, build, then `npm prune --omit=dev`; start `node`

- `src/server.js` with `systemd` /PM2. Do not invoke an npm prestart build after pruning Vite.
7. Deploy the Node application as identical instances behind the ALB; expose only port 3000 to the proxy security group.
  8. On the Python Analytics instance, create a virtual environment, install the pinned `06Code/python-analytics-api/requirements.txt` and start Uvicorn on port 8000 under a process manager.
  9. Configure Nginx/ALB routing, ACM HTTPS, HTTP→HTTPS, HSTS and exact production origins.
  10. Configure ALB health checks: `/api/v1/health/live` for process liveness and `/api/v1/health/ready` for traffic readiness. Treat `503` readiness as unavailable without expecting internal error details.
  11. Verify auth/session/CSRF, branch scope, roster finalization, finance reversal, reports, audit filters, `/api/analytics/v1/health`, cache headers and private-page redirects.
  12. Reconcile attendance, outstanding balances and report totals, then keep monitoring readiness, 5xx rate, latency and data-quality alerts before deleting the migration snapshot.

## HTTPS Recommendation

Use ACM certificates on ALB. Redirect all HTTP traffic to HTTPS. Keep cookies Secure in production.

## Google OAuth Origins

For Google Sign-In, configure Authorized JavaScript origins with an HTTPS public domain:

- Local development: `http://localhost:5500` , `http://127.0.0.1:5500`
- Production: `https://app.americanlatinclass.edu` or another domain controlled by the team

Do not use raw public IP origins. Configure the organization domain with HTTPS first and allow only its exact origin.

## React Frontend Build on EC2

Before starting the production Express process, build the React + Vite frontend:

```
cd 06Code
npm ci
npm run db:generate
npm test
npm run frontend:build
```

```
npm prune --omit=dev
node src/server.js
```

The Vite output is `06Code/dist/frontend`. Express serves that directory and keeps `/api/v1` as the API prefix. `npm start` is convenient for local/source deployments because its prestart hook builds the frontend; a pruned production runtime must use its prebuilt artifact and start `node src/server.js`. If Nginx is used, configure the site root to `06Code/dist/frontend` and proxy `/api/v1` to the Node.js process.

For the current EC2 frontend instance, the reference Nginx configuration is versioned at `070ther/nginx-alc-frontend.conf`. It includes:

- `/api/v1/health/live` and `/api/v1/health/ready` proxy to the Node application and remain sanitized/public for infrastructure probes.
- `/api/analytics/v1/` proxy to Python Analytics API.
- `/api/v1/` proxy to the Node application.
- `/private/` no-cookie redirect to `/login.html?session=expired` plus `Cache-Control: no-store`.
- `/db-admin/` returns `404`; no Adminer/database console is exposed on the public site.
- `/assets/` immutable static cache with `Cache-Control: public, max-age=31536000, immutable`.
- `/` and `/login.html` HTML revalidation with `Cache-Control: no-cache, must-revalidate`.
- Organization HTTPS host (represented here as `https://academy.example.invalid`) for Google OAuth validation.

## Cache Management Deployment Checks

After deploying the latest code and reloading Nginx, verify cache policy headers through HTTPS:

```
curl -I https://academy.example.invalid/
curl -I https://academy.example.invalid/login.html
curl -I https://academy.example.invalid/assets/<built-asset-file>
curl -I https://academy.example.invalid/api/v1/auth/config
curl -I https://academy.example.invalid/api/v1/auth/me
curl -I https://academy.example.invalid/api/v1/health/live
curl -I https://academy.example.invalid/api/v1/health/ready
curl -I https://academy.example.invalid/api/analytics/v1/health
```

Expected policies:

- HTML pages: `X-Cache-Policy: html-revalidate`.
- Static assets: `X-Cache-Policy: public-static-immutable`.

- Auth config: `X-Cache-Policy: public-auth-config` .
- Anonymous/private API responses: `X-Cache-Policy: sensitive-no-store` .
- Node liveness: `200` ; readiness: `200` when persistence is available and sanitized `503` otherwise.
- Python health: `X-Cache-Policy: public-health-short` .

Authenticated memory-cache evidence can be validated by logging in, then repeating the same protected read twice:

```
curl -H "Authorization: Bearer <session-token>"
https://academy.example.invalid/api/v1/roles
curl -H "Authorization: Bearer <session-token>"
https://academy.example.invalid/api/v1/roles
```

The first response should include `X-Memory-Cache: MISS` ; the repeated response should include `X-Memory-Cache: HIT` .

## Python Analytics API EC2

Recommended setup:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
git clone https://github.com/dxmar24/AWD-30716-Quantum-Code.git
cd AWD-30716-Quantum-Code/06Code/python-analytics-api
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Create `.env` on the instance:

```
DATABASE_URL=<injected-private-database-url>
SESSION_SECRET=<same secret used by Node Application API>
ANALYTICS_AUTH_REQUIRED=true
ANALYTICS_CORS_ORIGINS=https://academy.example.invalid
```

Keep `ANALYTICS_AUTH_REQUIRED=true` in staging and production. The service validates the shared session and enforces student, teacher and branch resource scope before returning analytics.

Credential rotation, deployed `/db-admin/` verification and incident-log review are external operational actions; repository sanitization does not prove they were completed. Follow `03Documentation/system-hardening-and-business-rules.md` before production sign-off.

Start command:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Production should run it with `systemd` or PM2 and proxy it through the Frontend EC2 Nginx site:

```
location /api/analytics/v1/ {  
    proxy_pass http://<python-analytics-private-  
ip>:8000/api/analytics/v1/;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```